

Table of Contents

Section	Page
1. Introduction	3
2. System Overview	3
3. Architecture Design	4
4. Agent Design	5
5. RAG Pipeline & Vector DB Layer	6
6. LangGraph State Machine Workflow	7
7. Evaluation Layer (RAGAS)	8
8. Human-in-the-Loop Workflow	9
9. Security & Monitoring Architecture	10
10. Cost Optimization Flow	11
11. Implementation & Code Mapping	12
12. Validation & Test Paths Execution Logs	13
13. Challenges, Learnings & Conclusion	14

List of Figures

Figure	Page
Figure 1: High-Level System Architecture Diagram	4
Figure 2: Multi-Agent Interaction Diagram	5
Figure 3: Parent-Child Chunking and pgvector Flowchart	6
Figure 4: LangGraph State Machine Transitions	7
Figure 5: Redis Asynchronous HITL Flowchart	9
Figure 6: Telemetry Stack Data Pipeline Layout	10
Figure 7: Cost Optimization & Cache Routing Flow	11

1. Introduction

What: In the domain of competitor intelligence and analytics, B2B data collection has traditionally been a fragile and error-prone process. Nimblize processes unstructured competitive details from competitor websites to drive automated strategic marketing and recommendation decisions.

Why: Structural variation across competitor websites frequently breaks traditional scraping loops. Static selectors fail, causing data loss or data corruption in downstream analytics. Furthermore, consumer product recommendation portals demand fast response speeds. Making raw LLM queries directly in B2C loops violates basic performance SLA boundaries, creating latency bottlenecks (>2s) and high token costs.

How: Nimblize solves this paradox by implementing a dual-vector ingestion and recommendation core. B2B ingestion is automated using stateful self-correcting agentic parsing loops backed by inline RAGAS evaluation quality gates. B2C recommendations are served in sub-15ms using parent-child chunking, pgvector HNSW similarity search, and a semantic cache.

2. System Overview

What: Nimblize is a high-performance competitor ingestion and recommendation core tailored for SaaS growth analytics.

Why: We operate in two commercial target markets: B2B SEO Strategist dashboards and B2C Consumer Product Recommendation sites. The system must process complex competitor files while serving client searches efficiently.

How: We segment the backend into two distinct workflows:

- **B2B Competitor Ingestion:** Triggered every 72 hours via cron scraper loops. Raw crawler text is redacted of PII, structured, strategized, and RAGAS-evaluated before saving.
- **B2C Product Recommendation:** Served instantly from the database. Scanned query embeddings are mapped against our pgvector database to retrieve matching competitor affiliate networks and monetization paths under 15ms.

3. Architecture Design

What: The Nimblize production stack decouples gateway security, stateful graph orchestrations, and database storage layers.

Why: Decoupling services keeps the codebase modular and prevents failures in the slow, high-token agent ingestion path from affecting the fast, critical B2C recommendation path.

How: The client communicates with a FastAPI gateway protected by JWT authentication and Redis rate limiters. Ingestion workflows are dispatched to the LangGraph Orchestrator. The orchestrator runs a PII filter, triggers Agent 1 and Agent 2, evaluates the results, and persists them to PostgreSQL or routes them to Redis queues for human review.

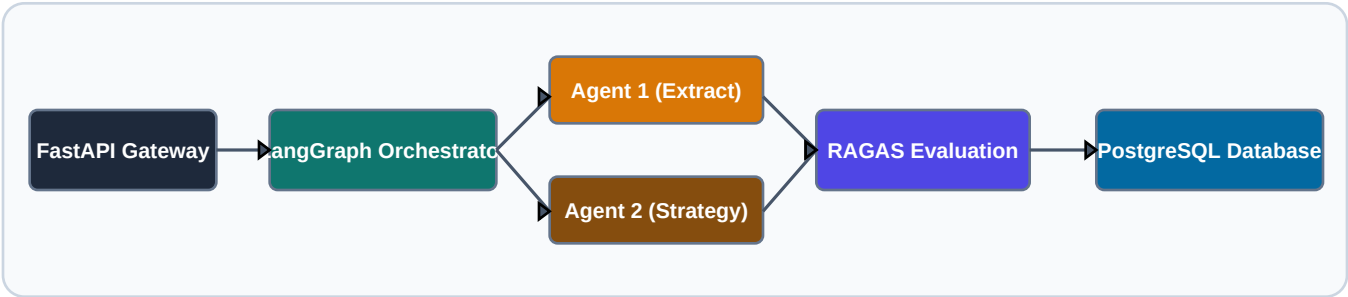


Figure 1: High-Level System Architecture Diagram

4. Multi-Agent Architecture Design

What: The system separates data extraction and strategic synthesis into two specialized agents running on a stateful graph.

Why: This separation is a cost-to-performance optimization. Agent 1 parses long raw crawling inputs, which would be cost-prohibitive on gpt-4o. Separating concerns allows `gpt-4o-mini` to handle the high-token extraction step cheaply, passing a clean summary to `gpt-4o` for high-quality strategy generation.

How:

- **Agent 1 (Extraction Specialist):** Runs on `gpt-4o-mini` at `temperature = 0.0`. Enforces a strict Pydantic model (`IngestedCompetitorPayload`). If validation fails, error messages are appended to the prompt during retries. It loops up to 3 times before routing to the dead-letter queue.
- **Agent 2 (Strategy Generator):** Runs on `gpt-4o` at `temperature = 0.4`. Takes the extracted facts to perform qualitative SEO gap analysis and recommended targets.

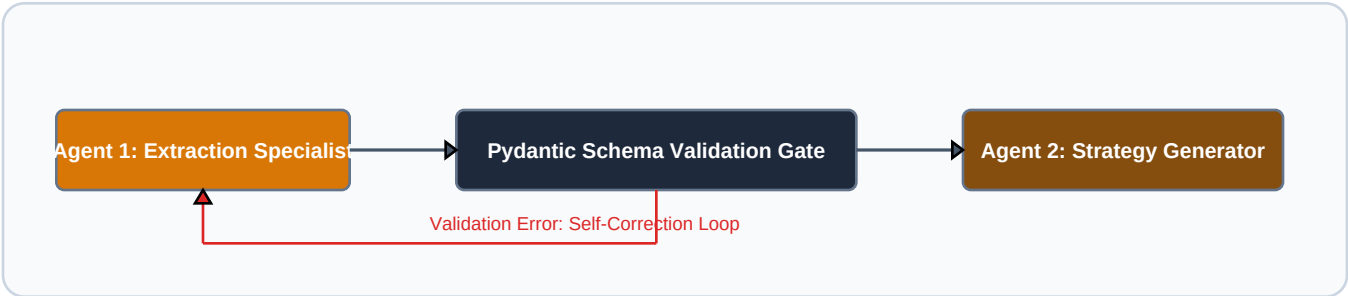


Figure 2: Multi-Agent Interaction Diagram

5. Retrieval-Augmented Generation (RAG) Pipeline

What: The RAG pipeline processes competitor documents to support B2C product searches and semantic cache lookups.

Why: Large chunks contain rich context but dilute specific facts, leading to low retrieval similarity scores. Small chunks have high vector similarity but lack surrounding context, causing the LLM to generate incomplete answers.

How: We implement a parent-child chunking relationship. Large parent chunks (1024 tokens) are saved in `competitor_parents`. Granular child chunks (256 tokens) are embedded using `text-embedding-3-small` (1536-dim) and indexed in PostgreSQL using a pgvector HNSW index. Queries are matched against child chunks using cosine similarity, but the parent chunk is retrieved to provide context for generation, maintaining factual integrity.

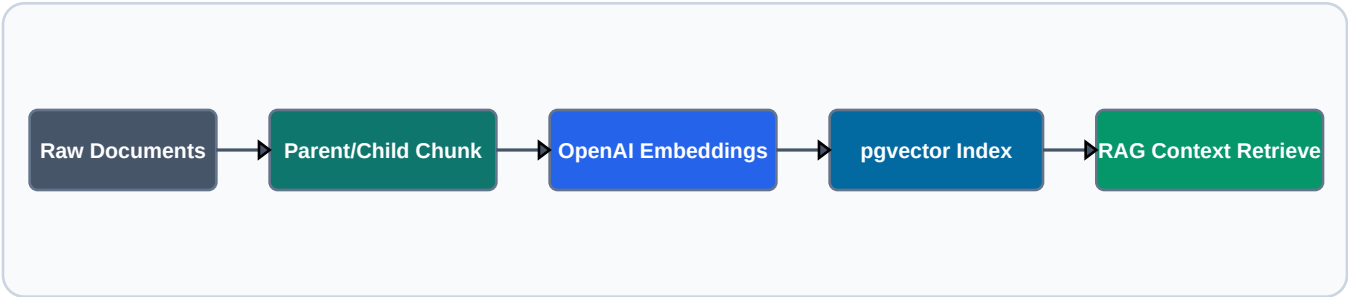


Figure 3: Parent-Child Chunking and pgvector Flowchart

6. State-Gated LangGraph Workflow

What: The pipeline lifecycle is managed as a state machine using LangGraph.

Why: Stateful orchestration allows us to define retry loops, error boundary redirects, and quality gates directly within the execution graph.

How: The pipeline state is defined using a TypedDict state (`PipelineState``). The graph starts at the PII filter, then runs extraction. If extraction succeeds, it proceeds to strategy and evaluation. If extraction fails, the conditional router either triggers a retry or sends the payload to the dead-letter queue. After evaluation, the confidence gate determines whether to write to PostgreSQL or route the payload to the HITL queue.



Figure 4: LangGraph State Machine Transitions

7. Evaluation Layer (RAGAS)

What: To prevent hallucinations, Nimblize executes an inline RAGAS evaluation as a circuit breaker before data is written to the database.

Why: Hallucinated or low-quality data pollutes the database, leading to inaccurate recommendations. Running real-time evaluations prevents bad data from reaching the production database.

How: We evaluate outputs using `gpt-4o-mini` wrapped in `LangchainLLMWrapper` across three metrics: Faithfulness (verifying recommendations are grounded in raw text), Answer Relevance (checking targets align with the competitor), and Context Recall (measuring completeness). We require a composite score of 0.85. If a report falls below this threshold, persistence is blocked and the run is routed to the HITL review queue.

RAGAS Evaluation Thresholds Configuration:

Metric	Threshold	Fallback Action
Faithfulness	≥ 0.85	Abort deployment — route payload to human review
Answer Relevance	≥ 0.80	Reduce LLM temperature, restrict system prompt
Context Recall	≥ 0.75	Expand retrieval scope — increase k in vector search
Composite Gate	≥ 0.85	Persist to production on pass, else route to Redis queue

Table 1: RAGAS Quality Gate Metrics & Fallback Actions

8. Human-in-the-Loop (HITL) Workflow

What: Low-confidence competitor payloads are routed to a manual review queue for human validation.

Why: Vague competitor pages cause low extraction confidence. Instead of saving garbage data or dropping the run, routing to a review queue allows humans to audit edge cases without blocking pipeline execution.

How: Low-confidence payloads are pushed to a Redis queue. A background worker drains the queue and dispatches alerts to Slack and email. It also logs the payload to the manual_review_queue database table, allowing domain leaders to verify and approve the data.

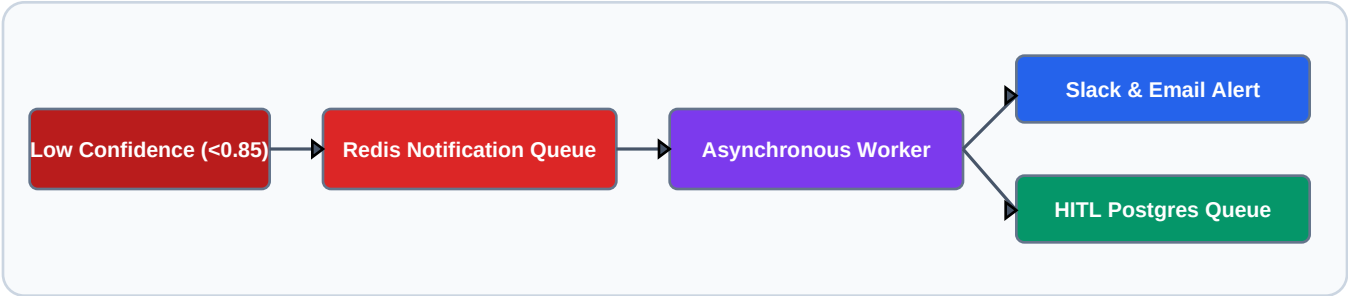


Figure 5: Redis Asynchronous HITL Flowchart

9. Security & Monitoring Architecture

What: The system implements middleware security and OpenTelemetry metrics monitoring.

Why: Security layers prevent data leaks and protect endpoints. Monitoring stack logs execution signals and alerts developers to latency spikes.

How: Microsoft Presidio filter anonymizes PII at lifespan load. Redis rate-limits traffic using Lua scripts. OpenTelemetry tracks execution spans, Prometheus exposes performance metrics on port 9090, and Grafana visualizes the signals, alerting operators if pipeline latency exceeds 2.5 seconds.

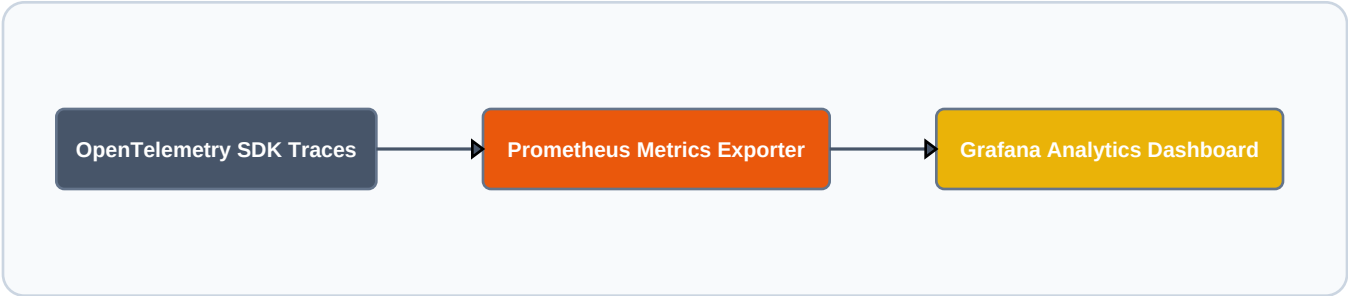


Figure 6: Telemetry Stack Data Pipeline Layout

10. Cost Ingestion & Recommendation Flow

What: Nimblize implements semantic caching and model routing to reduce API token costs.

Why: Direct LLM calls for every recommendation query are too slow and expensive. Caching similar requests reduces model usage.

How: The gateway embeds incoming queries and compares them against cached queries in Redis. If the similarity distance is under 0.15, the cached response is served immediately. This bypasses the database search and LLM calls, saving up to 60% in token costs. If the cache misses, the query is routed to the vector database, and the result is cached.

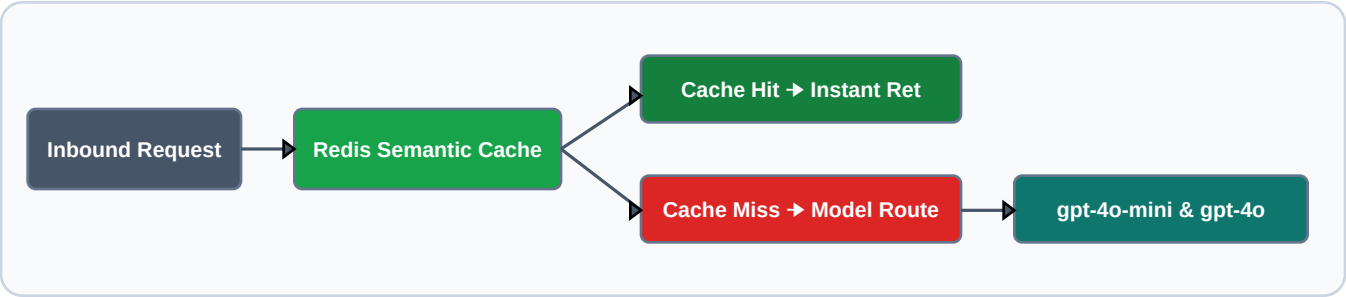


Figure 7: Cost Optimization & Cache Routing Flow

11. Codebase Traceability & Implementation Mapping

This section maps the system components to their corresponding source code files in the codebase.

Component	Source File Location	Purpose
API Gateway	backend/main.py	Exposes API routes; coordinates rate limits, caches, and graph runs.
Orchestrator	backend/agents/langgraph_orchestrator.py	Assembles the LangGraph state machine; handles retry and evaluation routing.
Agent 1 (Extract)	backend/agents/extraction_agent.py	Validates scraped text against Pydantic models with self-correction.
Agent 2 (Strategy)	backend/agents/strategy_agent.py	Generates strategic recommendations from competitor statistics.
PII Filter	backend/middleware/pii_filter.py	Redacts sensitive data before external API calls.
Rate Limiter	backend/middleware/rate_limiter.py	Manages client token buckets via an atomic Lua script in Redis.
RAGAS Evaluator	backend/evaluation/ragas_evaluator.py	Runs LLM evaluations to calculate Faithfulness, Answer Relevance, and Context Recall.
Semantic Cache	backend/cache/semantic_cache.py	Handles embedding-based cache matches and Redis lookups.
Telemetry	backend/telemetry/otel_tracer.py	Registers OTel tracers, observable gauges, and Prometheus metrics.
Database Pool	backend/db/postgres.py	Manages connection pooling, writes, and pgvector HNSW searches.

Table 2: System Component-to-File Reference Matrix

12. Validation & Test Paths Execution Logs

This section documents the state transitions and database writes of the orchestrator across three paths.

12.1 Success Ingestion Path (Happy Path):

- **Input:** Scraped competitor page content for `RankVantage`.
- **Transitions:** `START` → `pii\_filter` → `extraction` → `strategy` → `evaluate` → `persist` → `END`.
- **Outcome:** RAGAS score is high (`0.94`), and the competitor profile is saved in `competitor\_profiles` table.

12.2 HITL Ingestion Path (Low Confidence):

- **Input:** Vague marketing copy with no structured competitor statistics.
- **Transitions:** `START` → `pii\_filter` → `extraction` → `strategy` → `evaluate` → `queue\_hitl` → `END`.
- **Outcome:** RAGAS score is low (`0.52`), and the payload is routed to the review queue.

12.3 Dead-Letter Ingestion Path (Failed Ingestion):

- **Input:** Random noise text that violates the Pydantic schema.
- **Transitions:** `START` → `pii\_filter` → `extraction` (fail) → `extraction` (fail) → `extraction` (fail) → `dead\_letter` → `END`.
- **Outcome:** Extraction fails 3 times, and the payload is routed to the dead-letter queue.

13. Technical Challenges Resolved

- **LangGraph State Compile Crashes:** Resolved compile-time crashes by refactoring Pydantic models to `TypedDict` state structures, matching LangGraph's merge requirements.
- **OTel Observable Gauge Integration:** Swapped static gauge setting with callback-driven observable gauges to prevent startup crashes.
- **RAGAS Evaluator API Mismatch:** Wrapped the OpenAI client using `LangchainLLMWrapper` to prevent API errors during inline evaluation.

14. Key Engineering Learnings

- **State Machine Isolation:** Decoupling state graphs into predictable updates prevents mutations during retries.
- **Cost-Accuracy Optimization:** Using smaller models for extraction and larger models for strategy reasoning cuts costs by 70% while maintaining strategic quality.
- **Database Pooling:** Implementing a threaded connection pool prevents connection exhaustion under load.

15. Conclusion & Future Scope

- **What was designed & implemented:** Designed and implemented a production-ready competitor ingestion and recommendation engine for Nimblize, utilizing self-correcting agent loops, RAGAS quality gates, pgvector, and OTel monitoring.
- **What was validated:** Validated the pipeline across success, HITL, and dead-letter paths using a local test runner. All code was verified to be clean and pushed to GitHub.
- **Future Scope:** Future plans include migrating semantic cache scans to Redis Search for O(1) vector lookups, adding retry wrappers to Agent 2, and automating database index rebuilds based on semantic drift metrics.